CONTINUAL KNOWLEDGE LEARNING ON CONSUMER HARDWARE: ADAPTING A BYTE-LEVEL MIXTURE-OF-EXPERTS MODEL TO UNIVERSITY POLICY

A THESIS

Presented to the University Honors Program

California State University, Long Beach

In Partial Fulfillment

of the Requirements for the

University Honors Program Certificate

Phu Thien Quach

B.S., 2026, California State University, Long Beach

Fall 2026

I, THE UNDERSIGNED MEMBER OF THE COMMITTEE,

HAVE APPROVED THIS THESIS

CONTINUAL KNOWLEDGE LEARNING ON CONSUMER HARDWARE: ADAPTING A BYTE-LEVEL MIXTURE-OF-EXPERTS MODEL TO UNIVERSITY POLICY

BY

Phu Thien Quach

Ju Cheol Moon, Ph.D. (Thesis Advisor)
Department of Computer Engineering and Computer Science

California State University, Long Beach

Fall 2026

ABSTRACT

CONTINUAL KNOWLEDGE LEARNING ON CONSUMER HARDWARE:

ADAPTING A BYTE-LEVEL MIXTURE-OF-EXPERTS MODEL

TO UNIVERSITY POLICY

By

Phu Thien Quach

Fall 2026

Organizations that hold sensitive documents may prefer a language model that they can train and run on their own hardware. This thesis examines whether a small, continually learning model can absorb institutional knowledge on a single consumer graphics card and what that adaptation costs. Starting from a published checkpoint of mini-AGI, a byte-level mixture-of-experts model, the study continued training on question-and-answer text built from 312 public California State University, Long Beach (CSULB) policy documents, mixed with the model's earlier subjects, in a two-part pilot followed by an eight-hour run. The model was evaluated on 400 reworded policy questions with embedding cosine similarity, exact-match fact checks, and word overlap, using Claude Opus 5.5 as a cloud reference. Answering from memory alone, the adapted model's cosine similarity to the official answers rose from 0.543 to 0.693 under the primary embedding model, a gain confirmed by a second embedding model and by paired bootstrap intervals, and its answers were closer to the official wording than Claude's. The gain came with measurable forgetting on two public benchmarks and two earlier subjects. Local adaptation proved feasible, but its knowledge reflects recall of trained wording and must be weighed against lost general ability.

ACKNOWLEDGEMENTS

I would like to thank my thesis advisor, Dr. Ju Cheol Moon of the Department of Computer Engineering and Computer Science, for guidance throughout this project, and in particular for encouraging me to frame the work around privacy and to evaluate the model's answers by their semantic similarity to official text.

I am grateful to the University Honors Program at California State University, Long Beach, for the opportunity to carry out this research.

This study builds on mini-AGI, an open-source language model developed by Alexey Borsky and released under the MIT License. The developer's decision to publish the code, the training records, and a trained checkpoint made this work possible.

AI-based coding and writing assistants were used to help develop the evaluation scripts and to prepare drafts of the project documentation and of this manuscript. All experimental decisions, results, and conclusions were reviewed by the author, and every reported number was checked against the saved result files.

Finally, I thank my family and friends for their support and encouragement during my studies.

TABLE OF CONTENTS

Page

ACKNOWLEDGEMENTS..... iii

LIST OF TABLES..... vi

LIST OF FIGURES..... vii

LIST OF ABBREVIATIONS..... viii

CHAPTER

1. INTRODUCTION..... 1

    Background and Motivation..... 1
    Problem Statement..... 2
    Research Questions..... 3
    Scope and Approach..... 3
    Contributions..... 4
    Organization of the Thesis..... 4

2. BACKGROUND AND RELATED WORK..... 6

    Knowledge Stored in Model Parameters..... 6
    Catastrophic Forgetting and Continual Learning..... 7
    Mixture-of-Experts Language Models..... 7
    Byte-Level Modeling and Adaptive Computation..... 8
    Retrieval-Augmented Generation..... 9
    Evaluating Generated Answers..... 9

3. THE MINI-AGI MODEL..... 11

    Origin and License..... 11
    Byte-Level Input and Recurrent Core..... 11
    Expert Pool and Routing..... 13
    Paging Experts Across Storage Tiers..... 13
    Growth and Pruning of Experts..... 14
    Built-In Safeguards Against Forgetting..... 14
    The Base Checkpoint..... 15

CHAPTER Page

4. METHODOLOGY 17

Overview of the Study 17
Collecting the Policy Corpus 18
Extracting and Sectioning Text 19
Building Question and Answer Pairs 19
Training Data Mixture 21
Training Procedure 21
Evaluation Protocol 23

5. RESULTS 29

Training Run 29
Closed-Book Policy Knowledge 29
Reading With the Source Passage 31
Results by Question Kind 33
Error Panel and Illustrative Answers 34
Effects on Previously Learned Subjects 35
Earlier Pilot Evaluation 39

6. DISCUSSION 40

Answers to the Research Questions 40
Recall of Trained Answers Versus Understanding 41
The Cost of Adaptation 42
Local Adaptation Versus a Cloud Model 43
Limitations 44

7. CONCLUSION AND FUTURE WORK 46

Summary 46
Future Work 47

APPENDICES 49

A. PROMPT AND ANSWER FORMATS 50

B. CHECKPOINT AND REPRODUCIBILITY RECORD 53

REFERENCES 55

LIST OF TABLES

TABLE Page

1. Addresses Skipped During the Crawl, by Reason..... 18

2. Kinds of Policy Facts Used to Generate Questions..... 20

3. Data Lanes Built From the Policy Corpus..... 21

4. Stages of the Training Procedure..... 22

5. How Local Answers Ended..... 25

6. Checkpoints Evaluated in This Study..... 29

7. Closed-Book Scores on the 400 Evaluation Questions..... 30

8. Paired Bootstrap Intervals for Closed-Book Cosine Gaps..... 31

9. Gold-Passage Scores on the 400 Evaluation Questions..... 32

10. Paired Bootstrap Intervals for Gold-Passage Cosine Gaps..... 33

11. Closed-Book Qwen3 Cosine by Question Kind..... 33

12. Held-Out Loss on the Base Subjects by Checkpoint..... 36

13. Held-Out Loss on the CSULB Subjects by Checkpoint..... 37

14. Frozen Benchmark Results by Checkpoint..... 38

15. Increase in enwik9 Loss by Type of Piece..... 39

16. Checkpoint Folders and Core Weight Hashes..... 54

LIST OF FIGURES

FIGURE Page

1. Layers of the mini-AGI model..... 12

2. Overview of the study..... 17

3. Held-out loss on the base subjects across checkpoints..... 37

LIST OF ABBREVIATIONS

API	Application programming interface
BM25	Okapi Best Matching 25, a keyword-based ranking function
CSU	California State University
CSULB	California State University, Long Beach
F1	Harmonic mean of precision and recall (word-overlap score)
GPU	Graphics processing unit
MoE	Mixture of experts
PDF	Portable Document Format
PG19	Project Gutenberg books published before 1919 (benchmark)
RAG	Retrieval-augmented generation
RAM	Random-access memory (system memory)
SHA-256	Secure Hash Algorithm with a 256-bit digest
URL	Uniform resource locator (web address)
VRAM	Video random-access memory (graphics card memory)

CHAPTER 1

INTRODUCTION

Background and Motivation

Large language models have become a common way to ask questions of written material, but the most capable systems are operated by a small number of companies and are reached over the internet. An institution that wants such a system to answer questions about its own documents usually has two options. It can send the documents, or excerpts of them, to an external service whenever a question is asked, which is the idea behind retrieval-augmented generation (Lewis et al., 2020), or it can train a model of its own so that the knowledge is held in the model's parameters. The first option shares institutional material with a third party every time the system is used. The second option has traditionally required computing resources that most universities, departments, and small organizations do not have.

Privacy and control motivate the second option. Policies, internal procedures, and records often contain information that an organization would prefer not to send to an outside provider, and a model that runs entirely on local hardware keeps both the documents and the questions inside the organization. A local model also continues to work without a network connection and remains under its owner's control. These benefits matter only if a model small enough to train locally can actually learn new

institutional material, and only if it can do so without losing the general abilities it already had.

This thesis studies that question using mini-AGI, an open-source language model released by Borsky (2026). The model reads text one byte at a time, stores most of its parameters as separate expert files on disk, loads onto the graphics card only the experts that the current text needs, and is designed to keep learning from whatever it reads. These properties make it a natural candidate for continual, local adaptation on consumer hardware. Public policy documents from California State University, Long Beach (CSULB) serve as a stand-in for an institution's private documents. They have the structure and vocabulary of institutional text, but because they are public, they can be collected, shared, and inspected without raising privacy concerns of their own.

Problem Statement

Continued training of a pretrained neural network on new material risks catastrophic forgetting, in which performance on earlier material degrades as the network adapts (French, 1999; McCloskey & Cohen, 1989). Studies of knowledge injection also report that facts presented during training are not always recoverable when a question is phrased differently (Allen-Zhu & Li, 2023; Ovadia et al., 2024). It is therefore not obvious that a small model trained for a few hours on one consumer graphics card can learn to answer questions about a body of policy text, nor what that training will cost in the model's other abilities. The problem addressed in this thesis is to measure both sides of that trade: the policy knowledge gained through continued training and the general capability lost.

Research Questions

The study is organized around four research questions.

1. Knowledge acquisition: After continued training on CSULB policy text, can the model answer reworded policy questions from its parameters alone, without being shown any document?

2. Reading: When the source passage for a question is placed in the prompt, how closely does the adapted model's answer match the official policy text?

3. Comparison with a cloud model: How do the adapted model's answers compare with those of a frontier cloud model, Claude Opus 5.5 (Anthropic, 2026), when both are scored in the same way?

4. Cost of adaptation: How much does the adaptation change the model's performance on the subjects and public benchmarks that the original developer used to evaluate it?

Scope and Approach

The study does not train a model from random initialization. It continues training from the checkpoint that Borsky (2026) published, which had already read about 1.87 billion characters of general text. No retrieval system is part of the evaluated pipeline. When a passage is supplied to a model, it is the stored passage from which the question was generated, not the output of a search engine. Claude Opus 5.5 serves only as a reference point for comparison; it did not write the training data and did not grade any answer.

The evaluated questions are new wordings of facts whose answers appeared in training under other wordings. The knowledge result therefore measures whether the

model can recall trained policy content when a question is phrased in a new way; it does not measure understanding of policies that the model never read. The evaluation protocol, which fixed the checkpoints, the question sample, the prompts, the embedding models, and the metrics, was written and committed to the project repository before any measurement was taken. One amendment, which added the campus name to the reference model's prompt, was recorded before the affected re-run. Every number reported in Chapters 5 and 6 is taken from result files saved in the project repository.

Contributions

This thesis makes four contributions.

1. A reproducible pipeline that collects public university policy documents, converts them into training text with several wordings per fact, and continues training a byte-level mixture-of-experts model on a single consumer graphics card.

2. A closed-book evaluation of the adapted model on 400 held-out question wordings, scored with two embedding models, exact-match fact checks, and word overlap, together with chance-level and ceiling baselines.

3. A matched comparison with a frontier cloud model that uses the same questions and the same scoring.

4. A measurement of forgetting with the original developer's own frozen benchmarks, which shows where and when the adaptation degraded previously learned abilities.

Organization of the Thesis

Chapter 2 reviews related work on knowledge stored in model parameters, catastrophic forgetting, mixture-of-experts and byte-level models, retrieval-augmented

generation, and the automatic evaluation of generated answers. Chapter 3 describes the mini-AGI model and the checkpoint from which this study started. Chapter 4 presents the methodology, from data collection through training and the evaluation protocol. Chapter 5 reports the results, Chapter 6 discusses them in light of the research questions and the study's limitations, and Chapter 7 concludes and suggests future work. Appendix A gives the prompt and answer formats, and Appendix B records the checkpoints and other information needed to reproduce the study.

CHAPTER 2

BACKGROUND AND RELATED WORK

Knowledge Stored in Model Parameters

Language models trained on large text collections retain a substantial amount of factual information in their weights. Petroni et al. (2019) showed that a pretrained model could complete factual statements well enough to be treated as a knowledge base without any fine-tuning. Roberts et al. (2020) studied closed-book question answering, in which a model answers with no access to external documents, and found that performance improved with model size and was competitive with systems that retrieved supporting text. The closed-book setting is the main condition in this thesis: the adapted model receives only the question and must answer from what training placed in its parameters.

Knowledge that appears in training is not always retrievable under a different phrasing. In controlled experiments with synthetic biographies, Allen-Zhu and Li (2023) found that a model could memorize training text yet fail to extract the facts it contained, and that presenting the same knowledge in several different forms during training made extraction more reliable. Ovadia et al. (2024) reported a related result for knowledge injection by fine-tuning: language models struggled to learn new facts from a single presentation, and exposure to several variations of the same fact helped. These findings informed the decision, described in Chapter 4, to generate several wordings for every policy fact.

Catastrophic Forgetting and Continual Learning

When a neural network is trained on new material, the weight changes that fit the new material can overwrite those that supported earlier behavior. McCloskey and Cohen (1989) described this effect as catastrophic interference, and French (1999) reviewed its causes and early attempts to reduce it. Later methods constrain the update itself; elastic weight consolidation, for example, slows learning on the weights that were most important for earlier tasks (Kirkpatrick et al., 2017).

For language models, Jang et al. (2022) framed continual knowledge learning as three simultaneous goals: retaining time-invariant knowledge, updating outdated knowledge, and acquiring new knowledge, and they measured how much knowledge was forgotten relative to how much was gained. Ibrahim et al. (2024) studied the continued pretraining of large language models on new data, measured forgetting as an increase in validation loss on the original data, and found that replaying a small fraction of earlier data substantially reduced it. This thesis adopts the same two-sided view. It measures knowledge gained on CSULB questions and knowledge lost as a change in held-out loss on the subjects the model had already learned.

Mixture-of-Experts Language Models

A mixture-of-experts layer replaces a single feed-forward network with many smaller expert networks and a router that sends each input to a few of them. Shazeer et al. (2017) showed that such sparsely gated layers could greatly increase model capacity while keeping the computation per input roughly constant. Fedus et al. (2022) simplified routing in the Switch Transformer and introduced an auxiliary load-balancing loss that

discourages the router from sending most inputs to a small number of experts. The mini-AGI model uses a variant of that balancing loss, as described in Chapter 3.

Sparse activation also matters for memory. Because only the selected experts take part in a given computation, the remaining experts do not need to occupy the graphics card. mini-AGI relies on this property to keep most of its parameters on disk.

Byte-Level Modeling and Adaptive Computation

Most language models split text into subword tokens drawn from a fixed vocabulary. Byte-level models instead read raw bytes, which removes the tokenizer and allows a model to read any text without a vocabulary built in advance. Xue et al. (2022) showed that byte-level models can be competitive with token-based models. Long-context byte-level models are commonly evaluated on the PG19 collection of books (Rae et al., 2020), and language models can also be compared as compressors by the number of bits per byte they need to encode held-out text, including a slice of the enwik9 Wikipedia dump (Delétang et al., 2024; Mahoney, n.d.). The original developer of mini-AGI uses these measures, and this thesis adopts them for its analysis of forgetting.

Adaptive computation allows a network to spend more processing on difficult inputs. Graves (2016) introduced adaptive computation time for recurrent networks, and Banino et al. (2021) proposed PonderNet, in which a learned halting probability determines how many steps the network takes. mini-AGI applies a recurrent block repeatedly and uses a PonderNet-style halting head to decide when each character has been processed enough.

Retrieval-Augmented Generation

Retrieval-augmented generation combines a language model with a search component that finds relevant documents and places them in the prompt (Lewis et al., 2020). Retrieval keeps knowledge outside the model, so documents can be updated without training. Ovadia et al. (2024) compared retrieval with unsupervised fine-tuning for knowledge injection and found that retrieval generally performed better. Keyword ranking functions such as Okapi BM25 remain common retrieval baselines (Robertson & Zaragoza, 2009).

Retrieval is a strong alternative, but it answers a different design question from the one examined here. When retrieval is combined with a cloud model, every question sends document excerpts to an external provider. This thesis evaluates the parametric option, in which knowledge is learned into a locally trained model. It uses a supplied source passage only as a secondary condition that measures how well the model reads, and it reports a cloud model's performance as a reference rather than as part of the system.

Evaluating Generated Answers

Generated answers are difficult to score automatically. Word-overlap F1, which counts the words shared by a predicted answer and a reference answer, has long been used for reading comprehension (Rajpurkar et al., 2016), but it rewards copying and penalizes correct paraphrases. Embedding-based measures compare the meaning of two texts by encoding each as a vector and computing the cosine of the angle between the vectors. Sentence-BERT made such sentence embeddings practical (Reimers & Gurevych, 2019), and Aynetdinov and Akbik (2024) proposed SemScore, which scores a

response by its embedding similarity to a reference response using the all-mpnet-base-v2 sentence encoder, a model built on MPNet (Song et al., 2020). They reported that this simple measure agreed with human judgments better than several more complex automatic metrics. More recent embedding models, such as the Qwen3 Embedding series (Zhang et al., 2025), are trained for a wide range of similarity and retrieval tasks.

Embedding similarity has a practical weakness for factual answers: two sentences that differ only in a number, a date, or the name of an office can receive very similar embeddings, so a fluent answer with a wrong fact may still score highly. For this reason, the evaluation in this thesis pairs cosine similarity with exact-match checks on policy numbers, dates, and URLs, and it reports chance-level and ceiling baselines so that cosine values can be interpreted. Differences between systems are tested with paired bootstrap resampling (Efron & Tibshirani, 1993), following recommendations for significance testing in natural language processing (Dror et al., 2018).

CHAPTER 3

THE MINI-AGI MODEL

Origin and License

mini-AGI is an open-source project by Alexey Borsky, released under the MIT License (Borsky, 2026). Its stated goal is a language model that can be trained from scratch on a single graphics card with 8 GB of memory, that keeps learning from everything it reads, and that stores its weights as ordinary files on disk. The developer describes the model as a small, toy-level system rather than a frontier model, and this thesis treats it in the same way: as a platform for studying local, continual adaptation, not as a competitor to large commercial models. This study used the developer's code and published checkpoint without rewriting the model. The CSULB tools, data, training records, and evaluation files were added on top of the original repository.

Byte-Level Input and Recurrent Core

The model reads text as a sequence of byte values, so its input alphabet consists of the 256 possible bytes plus a small set of marker symbols rather than a learned vocabulary. Figure 1 shows its layers. Each character first passes through two dense prelude blocks, each consisting of masked multi-head self-attention (Vaswani et al., 2017) followed by a feed-forward network. The resulting representation then enters a single recurrent block that can be applied up to 24 times with the same weights at every application. At each application, an adapter combines the block's previous state with the

embedded input, which keeps the repeated computation anchored to the text being read. Attention uses eight heads and rotary position embeddings, which encode positions without learned parameters (Su et al., 2024), and the context window is 4,096 characters.

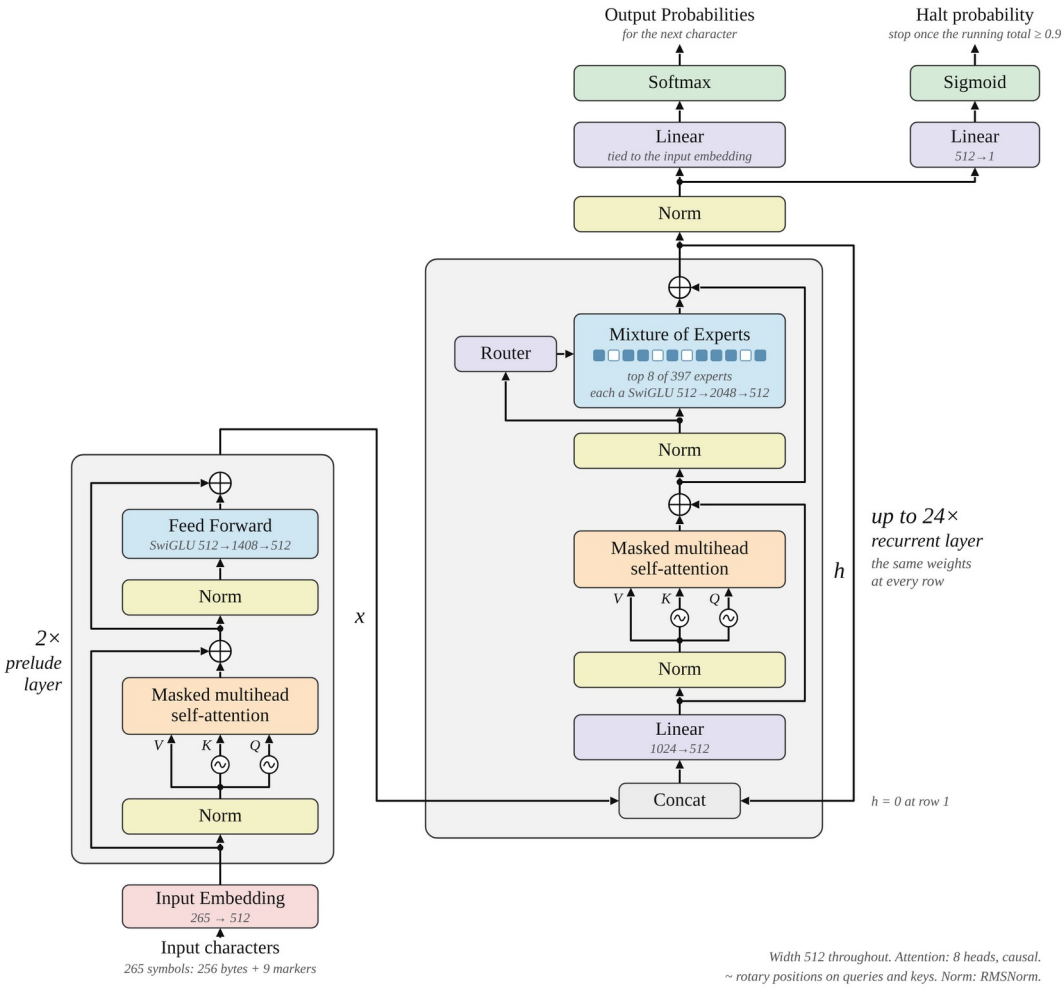


FIGURE 1. Layers of the mini-AGI model. Two prelude layers process each character once; the recurrent layer, which contains the mixture of experts, is applied up to 24 times with the same weights, and a halting head decides when to stop. Adapted from mini-AGI by A. Borsky (2026), MIT License.

The number of applications is decided separately for each character by a halting head trained with the PonderNet recipe (Banino et al., 2021). Easy characters stop after a single application, and difficult characters continue for more. Once a character halts, its

experts stop running, and later characters read its final state. The same procedure is used during training and during generation, so the model reads and writes through a single code path.

Expert Pool and Routing

Inside the recurrent block, the feed-forward computation is a mixture of experts. Each expert is a small SwiGLU feed-forward network (Shazeer, 2020). Every time a character passes through the recurrent block, a router selects its top eight experts. Because routing happens at every application of the block, one character can draw on many different experts over its successive applications. No expert is assigned to a subject in advance; the division of labor emerges from training.

During training, the loss includes a load-balancing term adapted from the Switch Transformer (Fedus et al., 2022). Instead of charging each expert for its share of the current batch, mini-AGI keeps a running share of the recent training passes that admitted each expert, so that use is balanced across texts rather than within a single text. The developer reports that this term substantially increased the number of experts that the router used on held-out text (Borsky, 2026).

Paging Experts Across Storage Tiers

Every expert is stored as a separate file that contains its weights and its own Adam optimizer state (Kingma & Ba, 2015). The model uses three storage tiers: all experts on disk, a cache of recently used experts in system memory, and a fixed number of resident experts on the graphics card. In the configuration used here, 32 experts are resident at a time. For each forward pass, every character ranks the entire pool with the router and requests its top eight experts. Those requests are pooled across the text, the 32

experts the text has requested most are loaded onto the card, and characters then route among the admitted experts only.

This design separates the total number of parameters from the memory of the graphics card. The base checkpoint used in this study held 397 experts and about 1,248.9 million parameters in total, of which about 100.7 million were on the graphics card at any time. Because each expert's optimizer state travels with the expert, an expert is always updated with its own history rather than with the history of whatever expert previously occupied the same slot.

Growth and Pruning of Experts

The expert pool changes size during training. New experts are created by recombination, which takes whole hidden units from several existing experts, and they begin with a small gate so that they change little until texts start to request them. Growth is allowed only when several conditions hold, including available disk and memory, evidence that recently added capacity is being used, and no sign that training loss and held-out loss are separating. Experts that no text has requested for a long time are pruned, and their files are deleted (Borsky, 2026). The pool contained 397 experts in every checkpoint evaluated in this study.

Built-In Safeguards Against Forgetting

Three mechanisms in mini-AGI are intended to limit forgetting. First, the shared trunk, which consists of the embeddings, attention layers, routers, and halting head, learns at one tenth of the experts' learning rate. Second, the model reads its training subjects in rotation, moving to a different subject after 32,768 characters (eight windows of 4,096 characters), so that no subject is read alone for a long stretch. Third, the

learning rate is not set by a fixed schedule. It is the product of a base rate and a plasticity scale that a controller adjusts after each held-out evaluation, easing it by at most 0.4% per check when held-out loss is not clearly improving and never lowering it below 1% of the base rate.

Borsky (2026) tested the first two mechanisms with a deliberately severe probe in which the model read 1,048,576 consecutive characters of PG19, a domain it had not seen, and nothing else. With the trunk at one tenth of the expert rate, the developer reports that the model retained 97.30% of its performance on the eight subjects it already knew, measured relative to chance, compared with 74.12% and 74.25% for configurations in which the trunk learned at the full rate. When PG19 was instead interleaved as a ninth subject, the reported retention was 100.13%, meaning that nothing was forgotten. These results indicate that the model is designed to absorb a new domain with limited damage when the domain is mixed with earlier material at a moderate learning rate. As Chapter 4 explains, the training in this study departed from those conditions during its first hour.

The Base Checkpoint

The starting point for this study is the checkpoint that Borsky (2026) published as an undertrained snapshot, referred to in this thesis as the base checkpoint. Its manifest records training step 914,508 and 1,872,164,864 characters read. It contained 397 experts, and its plasticity scale was 0.0515, about one twentieth of the base learning rate. The developer's held-out evaluation set used in this study covers seven subjects: arithmetic, chat, chess, code, reasoning, stories, and Wikipedia. The base checkpoint was copied before training, and only the copy was trained. The original folder was never

modified; its manifest timestamp was checked after the pilot, after the eight-hour run, and after the evaluation.

CHAPTER 4

METHODOLOGY

Overview of the Study

Figure 2 summarizes the study. Public policy pages and documents were collected from the CSULB website, converted to clean text, divided into sections, and turned into question-and-answer training text. The base checkpoint was then trained on that text, mixed with general subjects, in a two-part pilot followed by an eight-hour run. Finally, checkpoints from the run were evaluated in two ways: by answering 400 held-out policy questions and by the original developer's frozen next-byte benchmarks. The training and evaluation records each report a single NVIDIA graphics card with 24,564 MiB of memory, and the evaluation record identifies it as an NVIDIA GeForce RTX 4090.

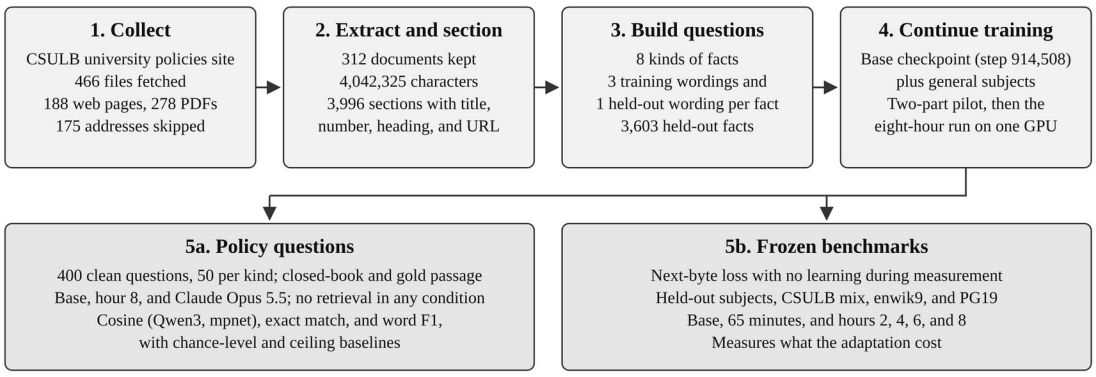


FIGURE 2. Overview of the study. The policy corpus was collected, sectioned, and converted into training text; the base checkpoint was trained on it; and the checkpoints were evaluated two ways.

Collecting the Policy Corpus

The crawler started from the CSULB university policies index (https://www.csulb.edu/university-policies) and followed links three levels deep: from the index to section pages, and from section pages to individual policy pages and the PDF files they linked. It also followed a short list of policies linked from the site footer, such as the nondiscrimination, privacy, accessibility, and Title IX policies and the annual security report. The crawler read each host's robots.txt file and did not fetch disallowed addresses. It honored the crawl delay of 10 seconds that www.csulb.edu publishes, capped each download at 60 MB, skipped images, spreadsheets, slides, archives, calendar files, and video and social-media hosts, and did not attempt to pass login pages. Word and RTF files were kept only when the listing page did not already contain the policy text.

The crawl fetched 466 files: 188 web pages and 278 PDF documents. It skipped 175 addresses, which are listed by reason in Table 1. Most skipped files were duplicate office copies of policies whose text already appeared on the corresponding web page.

TABLE 1. Addresses Skipped During the Crawl, by Reason

Reason	Count
Duplicate office file; the web page already contained the text	158
Page not found (HTTP 404)	5
Host disallowed by robots.txt	4
Access blocked (HTTP 403)	2
Login required	2
Web page returned instead of the PDF	2
Empty catalog response	1
Video link	1
Total	175

Extracting and Sectioning Text

For web pages, the extractor kept the main content block and removed scripts, styles, and navigation. PDF files were converted with pdftotext in reading order. Its layout mode was not used, because the spaces it adds to pad lines would have been copied by the model. Hyphenation across line breaks and stray carriage returns were removed. Documents shorter than 150 characters were dropped, and near-duplicate documents were detected with six-word shingles: when two documents shared more than 80% of the shorter document's shingles and their lengths were within a factor of two, only one copy was kept, with preference given to the web page because it carries the public address.

Each kept document was cut into sections at its headings, with each section between 120 and 1,500 characters long. Every section carried the document title, policy number, heading, URL, and responsible office. The result was 312 documents containing 4,042,325 characters, divided into 3,996 sections.

Building Question and Answer Pairs

A fact is one statement made by a policy, such as the content of a named section, a term that the policy defines, or the person who approved it. Table 2 lists the eight kinds of facts. For each fact, the corpus builder generated several question wordings with the same answer. Up to three wordings were placed in the training data, and one further wording was held out for evaluation. Generating several wordings per fact follows the evidence, reviewed in Chapter 2, that varied presentations of the same knowledge make it easier to retrieve under a new phrasing. In total, 3,603 facts received a held-out wording.

TABLE 2. Kinds of Policy Facts Used to Generate Questions

Kind	What the question asks
Topic	What a named section of a policy says
Definition	A term that a section defines
Overview	What the whole policy is about, from its first substantive section
Number	Which policy statement number the policy has
Date	The date on the policy
Supersedes	Which older policy it replaces
Approval	Who approved it
Where	The responsible office and the policy's web address

Every answer follows the same template so that the model can learn to cite its source. An answer begins with "According to," followed by the policy's title or policy statement number and the section heading, gives an excerpt of the policy text, and ends with a line that gives the source URL. A second copy of each training question was written in an open-book form, with the source passage placed in front of the question under a short header that gives the policy title, section, date, and source URL. Training, evaluation, and the project's chatbot all use the same function to build this prompt, so the bytes match exactly. The formats appear in Appendix A.

Table 3 lists the resulting data lanes. The open-book lane is the largest, which means that much of the model's CSULB training consisted of reading a passage and answering from it.

TABLE 3. Data Lanes Built From the Policy Corpus

Lane	Content	Size
Policies	The 312 cleaned documents	4.04 MB
Closed-book questions	Questions and answers without a passage	6.30 MB
Open-book questions	The same questions with the source passage	18.54 MB
Held-out closed-book	Held-out wordings without a passage	2.12 MB
Held-out open-book	Held-out wordings with the passage	6.28 MB

Note: The first three lanes were used for training. The two held-out lanes were used only to measure held-out loss.

Training Data Mixture

mini-AGI gives each training subject an equal share of reading. If CSULB text had been the only subject, the model would have spent the run repeating a few tens of megabytes of text. General subjects from the developer's corpus were therefore included alongside the three CSULB lanes: chat (about 443 MB), reasoning (about 144 MB), chess (about 126 MB), code (about 108 MB), and Wikipedia (about 55 MB). The developer's stories and arithmetic subjects exist in the local corpus only as very small files. They were included in the first 65 minutes of the pilot and then removed, as described below, but they remained in the held-out evaluation sets throughout.

Held-out loss during training was measured on a mixture that combines the developer's seven held-out subjects with the held-out CSULB questions in closed-book and open-book form. Because every subject in this mixture counts once in its average, a drop in CSULB loss is diluted by the other subjects.

Training Procedure

The base checkpoint was copied to a separate training folder. The first launch kept the base checkpoint's plasticity scale and failed on its first step because an expert

file was missing. No checkpoint from that attempt was kept, and the folder was copied again from the base checkpoint. Table 4 summarizes the stages that followed.

TABLE 4. Stages of the Training Procedure

Stage	Duration (min)	Subjects	Base learning rate	Final step
Pilot, part 1	65.6	10	0.0001	923,915
Pilot, part 2	55.1	8	0.00005	931,303
Eight-hour run, first launch	70.7	8	0.00005	940,860
Eight-hour run, resumed	410.1	8	0.00005	995,953

Note: Both parts of the pilot reset the plasticity scale to 1.0; the eight-hour run did not. The checkpoint at the end of pilot part 1 was kept as the 65-minute checkpoint. The checkpoints at steps 931,303 and 940,860 were later overwritten by continued training.

The first part of the pilot ran for 65.6 minutes with ten subjects, including stories and arithmetic, at a base learning rate of 0.0001. It reset the plasticity scale from 0.0515, about one twentieth of the base rate, to 1.0, so the model trained at its full base rate. CSULB loss fell quickly, but the training log showed that by the 64-minute sample, stories held-out loss had risen from 0.484 to 1.169 and arithmetic from 0.584 to 0.783. Because these two subjects are very small, an equal share meant that the model reread them many times. This checkpoint was saved separately and was not trained further.

The second part of the pilot continued from that checkpoint for 55.1 minutes with eight subjects, without stories and arithmetic, at a base rate of 0.00005, again with the plasticity scale reset to 1.0. Held-out loss on the training mixture fell from 0.6947 to 0.6292.

The eight-hour run continued from the end of the pilot without resetting the plasticity scale. Its first launch received a stop signal after 70.7 minutes, finished its current chunk, and saved a consistent checkpoint at step 940,860. Training then resumed

from that checkpoint for the remaining 410 minutes. A separate watcher process, which never signaled the trainer, saved snapshots at plan hours 2, 4, and 6 by hard-linking the training folder after the next checkpoint. Because the training code writes each file under a temporary name and then renames it, later training created new files instead of altering the linked ones. The run ended at step 995,953 after reading 112.83 million characters in 410.1 minutes of the resumed session, for a total of 480.8 minutes on the plan clock. Over its lifetime, the model had then read about 2,039.0 million characters. The plasticity scale ended at 0.9896, so the learning rate stayed close to its full value throughout the run.

Evaluation Protocol

The evaluation was designed after the training run and written down before any measurement. The protocol fixed the checkpoints, the random seed and question sample, the four prompts, the two embedding models, and the primary and secondary metrics, and it was committed to the repository together with the question files. The measurement tools from the original project gained only options to set an output path, to choose the held-out folder, and to force a fresh measurement. With those options omitted, each tool behaves as before, and the measurement arithmetic was not changed. All measurements used a single version of the code.

Two families of measurement follow from the research questions. Policy-question answers address the first three questions, and frozen next-byte benchmarks address the fourth. Answers were generated for the base checkpoint and the hour-8 checkpoint. The benchmarks were run on the base checkpoint, the 65-minute checkpoint, and the checkpoints at hours 2, 4, 6, and 8.

Question Sample

A held-out question was treated as clean when its exact wording did not appear anywhere in the closed-book training file. Because the same wording can be generated for two different facts, 1,089 of the 3,603 held-out wordings collided with a training question, which left 2,514 clean questions. From these, 400 questions were drawn with a fixed seed (20261029): 50 of each of the eight kinds, with at most two questions from the same document within a kind. A further 20 clean questions, drawn from those that remained, were reserved for developing the reference model's prompt and were not scored. Of the 400 questions, 279 have a policy statement number, all 50 date questions have a document date, and all 400 have a URL. Eleven of the 400 also appeared in an earlier 60-question pilot evaluation, and the sample was not redrawn to remove them.

Answer Generation

Each question was asked in two conditions. In the closed-book condition, the model received only the question, in the same chat format used in training. In the gold-passage condition, the model received the stored source passage for that question followed by the question, formatted by the same function used to build the open-book training data. No search was performed in either condition.

Local answers were generated with learning switched off and with greedy decoding, so a given checkpoint always produces the same answer, using the serving code's built-in repetition guard. Each answer could run for up to 800 generation steps, each usually one character, and was cut at the model's end-of-reply marker or at the start of a new conversational turn. The earlier pilot had used a limit of 360 characters, which stopped many answers before they reached their source line; the longer limit was chosen

so that answers could finish. In total, 1,600 local answers were written: 400 questions in two conditions for each of the two checkpoints. Table 5 shows how they ended. Most hour-8 answers ended on their own, whereas the base checkpoint often ran until the limit.

TABLE 5. How Local Answers Ended

Checkpoint	Closed-book, ended	Closed-book, hit limit	Gold passage, ended	Gold passage, hit limit
Base	204	196	266	134
Hour 8	382	18	388	12

Note: An answer "ended" when the model produced its end-of-reply marker before the 800-step limit.

Reference System

Claude Opus 5.5 (Anthropic, 2026) was queried through its programming interface, one question per request, with no conversation history, at the low effort setting, and without a temperature setting, which this model does not accept. It never received a local answer or a reference answer. Before the scored run, 20 development questions were used for one permitted format change: because the development answers ran longer than requested, both prompts were changed to ask for an answer of two to five sentences and to forbid headings and lists. The prompts were then frozen (Appendix A). The 400 questions were submitted in both conditions through the batch interface, and the first 50 were submitted a second time to measure how much Claude's answers vary between runs. All 900 replies ended normally, with no refusals and no replies cut off by the length limit.

After the first run, a review found that the closed-book prompt never told Claude that the questions concern CSULB, while 201 of the 400 question texts mention neither CSULB, Long Beach, nor California State University. The local model, trained only on

CSULB policies, does not need that context, but Claude had to guess the institution. An amendment recorded before the re-run added one sentence at the start of the closed-book prompt, stating that the questions are about policies of California State University, Long Beach. The 400 closed-book questions and the 50-question repeat were then run again with this second version, and all 450 replies ended normally. The gold-passage prompt was not changed, because the passage already names the policy and its CSULB source. Results for both closed-book versions are reported, and the second version is used for comparisons.

Scoring

The primary metric is the cosine similarity between an answer and the official reference answer for its question. Two embedding models were fixed in advance: Qwen3-Embedding-0.6B (Zhang et al., 2025) as the primary model and all-mpnet-base-v2 (Reimers & Gurevych, 2019; Song et al., 2020) as the secondary model. Both texts were encoded without an instruction, the vectors were normalized to unit length, and the cosine was computed as their dot product. A ranking of systems is reported only when both embedding models agree on it.

Before embedding, three kinds of boilerplate were removed from every text: a leading “According to” clause through its first colon, any line beginning with “Source:”, and chat markers. The adapted model was trained to produce exactly this template, so leaving it in place would have rewarded the template rather than the content. The answer and the reference were then cut to the length of that question’s cleaned reference, so that longer answers were not compared on different terms.

Three supporting measures were computed on the raw answer before any stripping. An answer received credit for the policy number when it contained the stored policy statement number, for the date when it contained the date string from the reference, which was scored only for date questions, and for the URL when it contained the full source URL. Questions without a policy number were left out of the number rate. Word-overlap F1 was computed with the project's existing function.

Two baselines make the cosine values interpretable. The chance level is the cosine between an answer and the reference for a different question, averaged over 10 random shuffles. The ceiling is the cosine between Claude's two answers to the same question on the 50 repeated questions, which shows how close two answers from a strong model are to each other. An error panel lists, for each system and embedding model, the answers in the top quarter of cosine scores whose policy number or date is missing, which identifies fluent answers that may still state a wrong fact. Differences between systems were tested with paired bootstrap resampling over the 400 questions with 10,000 resamples, and 95% intervals are reported.

Frozen Next-Byte Benchmarks

The original developer evaluates mini-AGI by next-byte prediction: the average loss, in nats per character or bits per byte, that the model incurs when it predicts held-out text that it did not train on. These measures were reused without modification to quantify forgetting. Frozen means that the weights only predict the text, and no learning step is taken during the measurement. Four measurements were made.

1. Held-out loss on the developer's seven held-out subjects (arithmetic, chat, chess, code, reasoning, stories, and Wikipedia), in nats per character.

2. Held-out loss on the training mixture, which adds the held-out CSULB questions in closed-book and open-book form.

3. Bits per byte on 2,000 pieces of 2,048 bytes drawn from the last 50,000,000 bytes of enwik9, a slice of English Wikipedia whose pages the model’s Wikipedia training data does not contain, scored in the manner of Delétang et al. (2024).

4. Bits per byte on the test split of PG19: 100 books containing 41,289,001 scored bytes, read in windows of 4,096 bytes.

Held-out loss and enwik9 were measured on all six checkpoints, and PG19 was measured on the base, 65-minute, and hour-8 checkpoints. Every measurement was forced to run fresh instead of reusing a logged value, and all 29 records were confirmed to be fresh. Before the measurements, the manifest step and the SHA-256 hash of the core weight file of each checkpoint were compared with the recorded values, and all six matched. The base and hour-8 checkpoints were each measured twice, and the largest difference between repeats was under 0.0001 nats. The original developer treats a difference of about 0.03 nats as the smallest change worth claiming, and the same threshold is used here.

CHAPTER 5

RESULTS

Training Run

Table 6 lists the checkpoints evaluated in this chapter. During the eight-hour run, training read about 4,800 to 4,900 characters per second, and held-out loss on the training mixture fell from 0.6055 at the start of the resumed session to 0.5743 at its end. In the training log, CSULB held-out loss fell from 0.268 to 0.090 for closed-book questions and from 0.306 to 0.141 for open-book questions between the start and the end of the eight-hour plan.

TABLE 6. Checkpoints Evaluated in This Study

Checkpoint	Training step	Characters read	Plasticity scale
Base	914,508	1,872,164,864	0.0515
65-minute	923,915	1,891,430,400	1.0
Hour 2	950,140	1,945,139,200	1.0
Hour 4	965,184	1,975,949,312	0.9965
Hour 6	982,163	2,010,722,304	0.9921
Hour 8	995,953	2,038,964,224	0.9896

Note: Characters read are totals over the model's lifetime, including the characters read before this study. Every checkpoint contained 397 experts.

Closed-Book Policy Knowledge

Table 7 reports the closed-book scores. With only the question as input, the hour-8 checkpoint's cosine similarity to the official answer was 0.693 under the primary embedding model and 0.645 under the secondary model, compared with 0.543 and 0.461

for the base checkpoint. Word F1 rose from 0.117 to 0.562. The hour-8 checkpoint included the correct policy number in 112 of the 279 answers to which one applied, the correct date in 12 of the 50 date questions, and the full source URL in 174 of 400 answers, whereas the base checkpoint did so in 22, 2, and 0 cases. Each system's cosine lies above its own chance level.

TABLE 7. Closed-Book Scores on the 400 Evaluation Questions

Measure	Base	Hour 8	Claude, campus named	Claude, campus not named
Cosine, Qwen3	0.543	0.693	0.642	0.601
Cosine, mpnet	0.461	0.645	0.585	0.559
Chance level, Qwen3	0.305	0.374	0.401	0.366
Chance level, mpnet	0.206	0.259	0.282	0.259
Word F1	0.117	0.562	0.194	0.174
Policy number correct	22 / 279	112 / 279	32 / 279	32 / 279
Date correct	2 / 50	12 / 50	4 / 50	4 / 50
Full URL	0 / 400	174 / 400	0 / 400	0 / 400

Note: Qwen3 is Qwen3-Embedding-0.6B and mpnet is all-mpnet-base-v2. "Campus named" is the amended prompt that states that the questions concern CSULB; "campus not named" is the original prompt.

Table 8 gives paired bootstrap intervals for the differences. The hour-8 checkpoint exceeded the base checkpoint by 0.150 under Qwen3 (95% interval 0.126 to 0.174) and by 0.184 under mpnet (0.157 to 0.212). Neither interval includes zero, and both embedding models rank the three systems in the same order: hour 8, then Claude, then the base checkpoint.

TABLE 8. Paired Bootstrap Intervals for Closed-Book Cosine Gaps

Comparison	Embedding	Mean difference	95% interval
Hour 8 minus base	Qwen3	0.150	0.126 to 0.174
Hour 8 minus base	mpnet	0.184	0.157 to 0.212
Hour 8 minus Claude, campus named	Qwen3	0.051	0.025 to 0.076
Hour 8 minus Claude, campus named	mpnet	0.060	0.029 to 0.092
Hour 8 minus Claude, campus not named	Qwen3	0.092	0.065 to 0.119

Note: Intervals come from 10,000 paired bootstrap resamples over the 400 questions.

Naming the campus in Claude's prompt raised its closed-book cosine by 0.041 under Qwen3 and by 0.026 under mpnet, and it raised its word F1 by 0.020, but it did not change Claude's counts of correct policy numbers, dates, or URLs. With the campus named, the hour-8 checkpoint remained ahead of Claude by 0.051 under Qwen3 and 0.060 under mpnet. Without the campus named, the Qwen3 gap had been 0.092. Naming the campus therefore narrowed the gap but did not reverse it.

The cosine between the hour-8 answer and Claude's answer to the same question was 0.577 under Qwen3 and 0.487 under mpnet, lower than the hour-8 checkpoint's similarity to the official reference (0.693). The adapted model's answers are therefore closer to the official policy wording than to Claude's answers. This is expected: the adapted model was trained on these reference answers under other wordings, whereas Claude paraphrases. For comparison, two closed-book answers from Claude to the same question had a cosine of 0.916 under Qwen3 and 0.895 under mpnet on the 50 repeated questions, and 0.896 and 0.887 for the version without the campus name.

Reading With the Source Passage

Table 9 reports the gold-passage condition, in which each model saw the stored source passage before the question. Claude's results in this condition come from the

original prompt, which was not changed. The hour-8 checkpoint's cosine similarity rose to 0.918 under Qwen3 and 0.909 under mpnet, compared with 0.634 and 0.577 for the base checkpoint and 0.792 and 0.760 for Claude. Its word F1 was 0.867, and it reproduced the full source URL in 353 of the 400 answers.

TABLE 9. Gold-Passage Scores on the 400 Evaluation Questions

Measure	Base	Hour 8	Claude
Cosine, Qwen3	0.634	0.918	0.792
Cosine, mpnet	0.577	0.909	0.760
Word F1	0.201	0.867	0.445
Policy number correct	25 / 279	236 / 279	278 / 279
Date correct	4 / 50	44 / 50	50 / 50
Full URL	0 / 400	353 / 400	79 / 400

With the passage in front of it, Claude included the correct policy number in 278 of 279 answers and the correct date in all 50 date questions, more often than the hour-8 checkpoint (236 and 44). Claude's lower URL count reflects a difference in format: the adapted model was trained to end every answer with a source line, whereas Claude's prompt asked it to mention a source only if the passage states one and limited its answer to two to five sentences. Table 10 shows that the hour-8 checkpoint exceeded the base checkpoint by 0.284 under Qwen3 and Claude by 0.127, and both embedding models again ranked hour 8 first, Claude second, and the base checkpoint third. The ceiling between Claude's two gold-passage runs was 0.954 under Qwen3 and 0.938 under mpnet, and agreement between the hour-8 and Claude answers was 0.765 and 0.726.

TABLE 10. Paired Bootstrap Intervals for Gold-Passage Cosine Gaps

Comparison	Embedding	Mean difference	95% interval
Hour 8 minus base	Qwen3	0.284	0.266 to 0.303
Hour 8 minus base	mpnet	0.332	0.310 to 0.353
Hour 8 minus Claude	Qwen3	0.127	0.110 to 0.143
Hour 8 minus Claude	mpnet	0.149	0.128 to 0.170

Results by Question Kind

Table 11 breaks the closed-book Qwen3 cosine down by question kind, with 50 questions of each kind. The hour-8 checkpoint was strongest on short, structured facts: policy numbers (0.926), approvals (0.886), web addresses (0.856), and superseded policies (0.813), and on these four kinds it was also ahead of Claude. Claude, with the campus named, scored higher than the hour-8 checkpoint on the three open-ended kinds: topic (0.583 against 0.458), overview (0.661 against 0.587), and definition (0.529 against 0.387).

TABLE 11. Closed-Book Qwen3 Cosine by Question Kind

Kind	Base	Hour 8	Claude, campus named	Claude, campus not named	Hour 8 word F1
Topic	0.407	0.458	0.583	0.563	0.345
Overview	0.480	0.587	0.661	0.615	0.387
Where	0.560	0.856	0.792	0.672	0.711
Date	0.483	0.629	0.632	0.534	0.527
Number	0.676	0.926	0.774	0.710	0.813
Definition	0.448	0.387	0.529	0.548	0.265
Approval	0.660	0.886	0.633	0.636	0.784
Supersedes	0.630	0.813	0.532	0.532	0.663

Note: Each kind contains 50 questions.

Definition questions are a clear weakness. On this kind, the hour-8 checkpoint's cosine was lower than the base checkpoint's under both embedding models (Qwen3 from 0.448 to 0.387, and mpnet from 0.368 to 0.293), even though its word F1 rose from 0.086 to 0.265. Both embedding models also placed Claude above the hour-8 checkpoint on definition questions (mpnet 0.521 against 0.293). On the other seven kinds, both embedding models placed the hour-8 checkpoint above the base checkpoint.

Error Panel and Illustrative Answers

Among the 100 hour-8 closed-book answers with the highest Qwen3 cosine, 36 were missing a policy number or date. In 19 of these, the missing number or date appeared in the official reference itself. In the other 17, the stored document had a number that the reference sentence did not use, so those answers did not omit anything that the reference contained. For Claude with the campus named, 78 of its top 100 closed-book answers were flagged, 66 of them for omitting a fact that the reference contains. In the gold-passage condition, none of Claude's top 100 answers was flagged, and all 21 flagged rows among the hour-8 checkpoint's top-ranked answers were of the second, harmless kind.

The demonstration question was selected by a rule recorded in the protocol amendment before selection: among topic, overview, and definition questions whose text names CSULB, take the one with the highest hour-8 closed-book cosine. The rule selected the question "Explain the CSULB 19-06 Range Elevation for Lecturer Faculty in short." The official reference states that Policy Statement 19-06, dated 5/16/2019, supersedes Policy Statements 00-11, 02-03, and 08-15, was recommended by the Academic Senate on March 21, 2019, and was approved by the president on April 18,

2019, and it gives the Academic Senate URL. The stored policy text confirms each of these claims. The hour-8 checkpoint reproduced the reference exactly, with a word F1 of 1.0, and stopped on its end-of-reply marker after 419 characters. The base checkpoint repeated the question as a numbered list, gave none of the facts, and ran to the generation limit. Claude, with the campus named, described range elevation as a lecturer's move to a higher salary range under the collective bargaining agreement, including an application and review, and that answer scored a cosine of 0.526 against the reference.

This example shows both the strength and the limit of the result. The adapted model recalled the policy's official record word for word, which is exactly what its training rewarded. Claude's answer, although less similar to the reference, explained what the policy does, which may be more useful to a reader who wants to understand the procedure.

A median example gives a less favorable view. Sorted by closed-book Qwen3 cosine, the two middle values among the 400 hour-8 answers are both 0.734. One of the two middle questions concerns the early-resolution section of the discrimination policy. The reference says that the complainant should contact the Human Resources Manager, but the hour-8 answer says that the complainant should consult the Executive Director, and it cites an Associated Students URL. The cosine is high, yet the office is wrong. This example shows why cosine similarity alone cannot certify that an answer is correct.

Effects on Previously Learned Subjects

Table 12 and Figure 3 report held-out loss on the developer's seven subjects. The mean across subjects moved from 0.6945 on the base checkpoint to 0.7028 at hour 8, but this small change in the mean hides large movements in opposite directions. Stories rose

from 0.476 to 0.905 nats per character and arithmetic from 0.657 to 0.970, increases of 0.430 and 0.314. Chat improved from 0.525 to 0.380 and chess from 0.960 to 0.413, decreases of 0.145 and 0.547. Code, reasoning, and the developer's Wikipedia held-out set changed by less than the 0.03-nat threshold.

TABLE 12. Held-Out Loss on the Base Subjects by Checkpoint

Subject	Base	65-minute	Hour 2	Hour 4	Hour 6	Hour 8
Arithmetic	0.657	0.793	0.812	0.839	0.915	0.970
Chat	0.525	0.504	0.452	0.411	0.386	0.380
Chess	0.960	0.446	0.414	0.415	0.413	0.413
Code	0.596	0.672	0.630	0.604	0.602	0.601
Reasoning	0.571	0.660	0.612	0.588	0.578	0.583
Stories	0.476	0.992	0.894	0.827	0.823	0.905
Wikipedia	1.077	1.158	1.108	1.075	1.073	1.067
Mean	0.6945	0.7465	0.7033	0.6796	0.6843	0.7028

Note: Values are nats per character on the developer's held-out set; lower is better. Repeated measurements of the base and hour-8 checkpoints gave the same means (0.6945 and 0.7028).

The timing of these changes points to the pilot. Stories had already risen to 0.992 at the 65-minute checkpoint, the end of the part of the pilot that read the stories and arithmetic files repeatedly at the full base rate. Stories partly recovered during the eight-hour run, falling to 0.823 at hour 6, before rising again to 0.905 at hour 8. Arithmetic behaved differently: it rose from 0.793 at 65 minutes to 0.970 at hour 8, continuing to worsen after it had been removed from the training mixture.

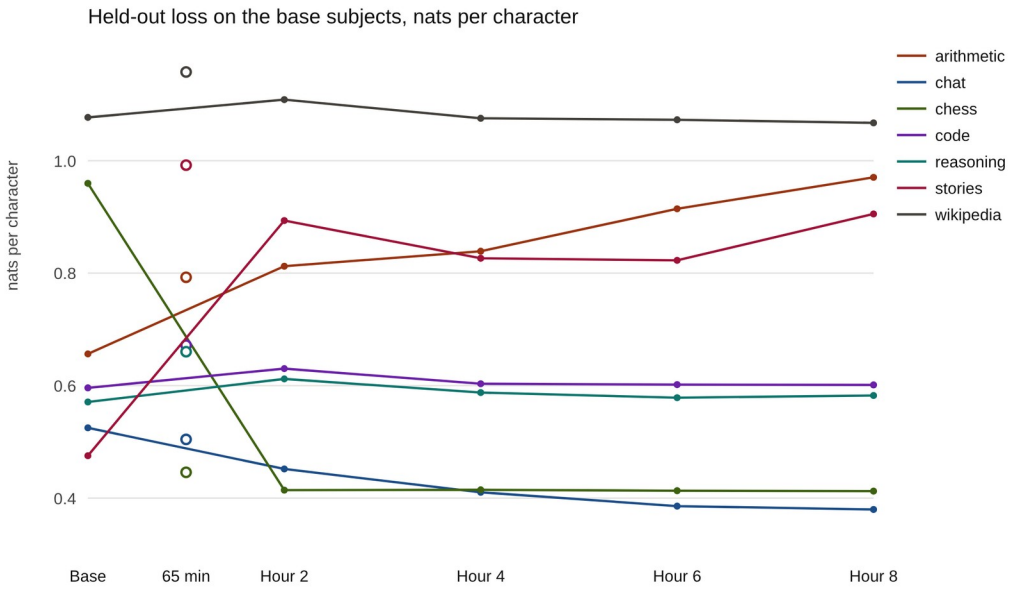


FIGURE 3. Held-out loss on the base subjects across checkpoints. Lines connect the base checkpoint with the checkpoints at hours 2 through 8; open circles mark the 65-minute pilot checkpoint, which used ten subjects, a higher base learning rate, and a reset plasticity scale. Values are nats per character; lower is better.

Table 13 shows the two CSULB held-out subjects. Closed-book CSULB question loss fell from 0.802 nats per character on the base checkpoint to 0.081 at hour 8, and open-book loss fell from 0.795 to 0.139. Most of the reduction had occurred by hour 2. On the training mixture, which includes these subjects, the mean fell from 0.7177 to 0.5710, and that drop comes from the two CSULB subjects.

TABLE 13. Held-Out Loss on the CSULB Subjects by Checkpoint

Subject	Base	65-minute	Hour 2	Hour 4	Hour 6	Hour 8
Closed-book questions	0.802	0.389	0.143	0.099	0.082	0.081
Open-book questions	0.795	0.434	0.246	0.184	0.160	0.139

Note: Values are nats per character on the held-out CSULB question wordings.

Table 14 reports the two public benchmarks. On enwik9, bits per byte rose from 1.3556 on the base checkpoint to 1.7359 at hour 8, with the worst value, 1.7790, at

hour 2. On PG19, bits per byte rose from 2.0578 to 2.2141. Both increases exceed the noise threshold. They contrast with the developer's own Wikipedia held-out set, which improved slightly, from 1.077 to 1.067, over the same checkpoints. This study records that difference between two Wikipedia measures without a full explanation.

TABLE 14. Frozen Benchmark Results by Checkpoint

Checkpoint	enwik9 (bits per byte)	PG19 (bits per byte)
Base	1.3556	2.0578
65-minute	1.6284	2.2864
Hour 2	1.7790	Not measured
Hour 4	1.6980	Not measured
Hour 6	1.6984	Not measured
Hour 8	1.7359	2.2141

Note: Lower is better. Repeated measurements gave 1.3556 (enwik9) and 2.0577 (PG19) for the base checkpoint and 1.7358 (enwik9) and 2.2141 (PG19) for the hour-8 checkpoint.

To locate the enwik9 increase, each of the 2,000 pieces was scored again for the base and hour-8 checkpoints with the loss of each piece saved, and the pieces were grouped by content (Table 15). Pieces containing wiki templates rose by 0.555 bits per byte, and pieces containing wiki tables rose by 0.550. Pieces with ordinary wiki markup, which make up 1,501 of the 2,000, rose by 0.331, which accounts for about two thirds of the total increase. The 26 pieces of plain prose rose by 0.114. Most of the damage therefore fell on Wikipedia's markup rather than on plain English text, although the prose group is small.

TABLE 15. Increase in enwik9 Loss by Type of Piece

Type of piece	Pieces	Increase (bits per byte)
Template	282	0.555
Table	191	0.550
Other wiki markup	1,501	0.331
Plain prose	26	0.114

Note: A piece is a table if it contains wiki table markup, a template if it contains at least two template openings and is not a table, other markup if it contains a wiki link, a heading, or a page tag, and plain prose otherwise. The increase is from the base checkpoint to the hour-8 checkpoint.

Earlier Pilot Evaluation

Before the protocol described in Chapter 4, a pilot evaluation scored 60 held-out questions with word F1 at several checkpoints. Its open-book condition used BM25 keyword search to choose the passage, and its answers were limited to 360 characters. In that pilot, closed-book word F1 rose from 13.4% on the base checkpoint to 31.6% after the pilot training and to 41.9% at hour 8. Open-book word F1, which combined retrieval with reading, rose from 17.7% to 51.6% and then to 57.0%. BM25 placed the correct policy document first for 80% of those 60 questions.

The pilot also revealed problems that the main evaluation was designed to fix. Fifteen of the 60 question wordings also appeared in training paired with a different document, 31 of the 60 hour-8 closed-book answers were exactly 360 characters long because the limit cut them off, and the search step mixed retrieval errors into the model’s score. The pilot also showed that closed-book answers often named a real CSULB policy and a real URL, and just as often named the wrong one, with Policy Statement 14-04 as a frequent wrong citation. The pilot results are reported here for completeness and are not compared directly with the main evaluation.

CHAPTER 6

DISCUSSION

Answers to the Research Questions

The first research question asked whether continued training could place CSULB policy knowledge into the model so that it answers reworded questions without a document. It did. Closed-book similarity to the official answers rose by 0.150 under the primary embedding model, the gain was confirmed by the secondary model and by paired bootstrap intervals that exclude zero, and the counts of answers that carried the correct policy number, date, or source URL rose sharply.

The second research question asked how closely the adapted model's answers match the official text when the source passage is supplied. With the passage, its cosine similarity reached 0.918 and its word F1 0.867, and it reproduced the full source URL in 353 of 400 answers. The open-book format was the largest part of the model's CSULB training data, and the model learned to copy the relevant excerpt and its citation reliably.

The third research question asked how the adapted model compares with a frontier cloud model under the same scoring. In both conditions, the adapted model's answers were closer to the official wording than Claude's, by 0.051 in the closed-book condition with the campus named and by 0.127 with the passage. Claude, however, scored higher on open-ended questions about topics, overviews, and definitions, and when given the passage it supplied the correct policy number and date more often. The

comparison therefore favors the adapted model on reproducing the official wording of structured facts, not on explaining policies in general.

The fourth research question asked what the adaptation cost. The cost was substantial. The model became worse at predicting enwik9 and PG19 text and at the stories and arithmetic subjects, while chat and chess improved, and code, reasoning, and the developer's Wikipedia held-out set were essentially unchanged.

Recall of Trained Answers Versus Understanding

The knowledge gain should be interpreted with care. Every reference answer in the evaluation was a training target for other wordings of the same fact, and only the question wording was new. A high cosine therefore shows that a new phrasing of a question retrieved the trained answer. This is a meaningful capability, because it is what "the knowledge is in the weights" means for a closed-book assistant, but it is not the same as answering questions about policies the model never read or reasoning about how two policies interact. The demonstration answer, which repeated the official record word for word, is an instance of exactly this kind of recall.

The pattern across question kinds supports this reading. The adapted model did best on short facts with a fixed form, such as numbers, approvals, and web addresses, which can be recalled as strings. It did worse on topic, overview, and definition questions, whose answers are longer excerpts, and on definition questions its cosine fell below that of the base checkpoint. The median example, in which a fluent answer named the wrong office, shows the corresponding risk: the model can produce confident, well-formed answers that attach a policy's language to the wrong fact. The earlier pilot

showed the same tendency, with closed-book answers that often cited a real CSULB policy that was not the correct one.

The Cost of Adaptation

The forgetting results show that the safeguards built into mini-AGI did not prevent damage under the conditions of this study. The developer's own probe found almost no forgetting when a new domain was interleaved with earlier subjects (Borsky, 2026). The first part of this study's pilot departed from those conditions in two ways. It reset the plasticity scale from 0.0515 to 1.0 with a base rate of 0.0001, and it gave the tiny stories and arithmetic files an equal share of reading, so the model reread them many times. Stories loss had already reached 0.992 at the end of that hour, compared with 0.476 on the base checkpoint, and enwik9 loss was highest at hour 2. Together, these observations point to the pilot's settings as the main cause, although this study did not run the controlled comparison that would confirm it.

Arithmetic continued to worsen after it was removed from training, rising from 0.793 at 65 minutes to 0.970 at hour 8. This is consistent with the general finding that knowledge not revisited during continued training tends to decay and that replaying a small amount of earlier data reduces forgetting (Ibrahim et al., 2024). Keeping stories and arithmetic in the mixture with more varied material, rather than removing them, is a natural remedy to test.

The enwik9 result deserves particular attention because the developer's own Wikipedia held-out set barely changed. The piece-level analysis shows that most of the enwik9 increase fell on templates, tables, and other wiki markup rather than on plain prose. One possibility is that the forms of markup common in the enwik9 slice were

poorly represented in the text the model read during this study, but the study did not test that explanation, and the prose group contained only 26 pieces.

Local Adaptation Versus a Cloud Model

The comparison with Claude Opus 5.5 frames the privacy argument. Claude is a far larger and more general model, and in this study it served as a reference rather than a competitor. Without the campus named, it sometimes could not identify the institution that a question referred to; for one question about where a department's policy is posted, its first-version answer treated the question as ambiguous and mentioned other universities. With the campus named and no document, it explained policies plausibly but matched the official wording less closely than the adapted model, and it never produced the full CSULB source URL. A small model trained locally on the institution's documents reproduced that wording and its citations more closely, without sending any document or question to an outside service when it was used.

That advantage has clear boundaries. Claude was better at open-ended explanation, and with the passage in its prompt it identified policy numbers and dates more reliably than the adapted model. In addition, the public policy documents used here stand in for private data, and the evaluation itself sent these public documents to a cloud provider to obtain the reference answers, which would not be appropriate for genuinely private material. For an institution whose documents cannot leave its control, the results suggest that local continual training is feasible on a single consumer graphics card, with two caveats: the model recalls trained wording rather than understanding policy, and its training must be managed to limit forgetting.

Limitations

Several limitations qualify these findings.

1. Trained answers. The reference answers were training targets under other wordings, so the evaluation measures recall under new phrasing rather than generalization to unseen policy content.

2. A single run. The study reports one training run. The pilot changed several settings at once, including the subjects, the base learning rate, and the plasticity scale, so the forgetting cannot be attributed to any one of them. Two intermediate checkpoints, the end of the pilot and the 70.7-minute checkpoint, were overwritten and could not be measured.

3. Scope of the answer evaluation. Answers were generated only for the base and hour-8 checkpoints, so the evaluation does not show how answer quality changed during the run. The sample contains 50 questions per kind, which limits the precision of comparisons between kinds.

4. Automatic scoring. Cosine similarity, exact match, and word F1 were not validated against human judgments in this study. Cosine can remain high when a number, office, or date is wrong, as the median example shows, and the error panel and exact-match counts only partly compensate for this weakness.

5. Reference model. Claude's answers vary between runs, as the ceiling measurements show, and Claude may have seen public CSULB pages during its own training. Its prompts asked for short answers without the trained citation template, which affects the comparison of URLs.

6. No abstention. The model was not trained to say that it does not know, so it is not designed to recognize questions that its training documents do not answer.

7. Starting point and data. The study continued training from a published checkpoint rather than training from scratch, and it used public documents as a stand-in for private ones.

CHAPTER 7

CONCLUSION AND FUTURE WORK

Summary

This thesis examined whether a small, continually learning language model can absorb an institution's policy knowledge on a single consumer graphics card, and what that adaptation costs. Starting from a published mini-AGI checkpoint, the study collected 312 public CSULB policy documents, turned them into question-and-answer text with several wordings per fact, and continued training the model in a two-part pilot followed by an eight-hour run. A protocol written before measurement scored 400 held-out question wordings with two embedding models, exact-match checks, and word overlap, compared the results with Claude Opus 5.5, and re-ran the original developer's frozen benchmarks on six checkpoints.

The adapted model learned the policy material in the sense that matters for a closed-book assistant. Given only a reworded question, its answers moved substantially closer to the official policy text, and with the source passage it reproduced the official wording and citation in most cases. Its answers were closer to the official wording than those of the cloud reference model, especially for structured facts, but less informative for open-ended questions. The adaptation also caused measurable forgetting on two public benchmarks and two earlier subjects, much of which appeared early, under pilot

settings that departed from the conditions in which the model's own safeguards had been shown to work.

The overall conclusion is that continual knowledge learning on consumer hardware is feasible and can keep institutional documents local, but it is not free. The knowledge gained is best described as recall of trained policy wording, and its value must be weighed against the general ability lost.

Future Work

Several directions follow from these results.

1. Controlled forgetting experiments that vary one setting at a time, namely the plasticity reset, the base learning rate, and the treatment of small subjects, together with replay of earlier data, to identify which choices caused the damage.

2. Evaluation on held-out documents, in which whole sections or policies are withheld from training, to measure whether the model can read and answer questions about material it has not memorized.

3. Human evaluation of a sample of answers, to validate the automatic metrics and to estimate how often the model gives confident but wrong answers.

4. Training the model to decline questions that its documents do not answer, and evaluating that ability.

5. Updating knowledge over time, such as replacing a superseded policy, which is the setting in which continual learning should offer the greatest advantage over retraining from scratch.

6. Combining the learned knowledge with a local retrieval step that keeps documents on the same machine, which may join the reliability of supplied passages with the privacy of local operation.

APPENDICES

APPENDIX A

PROMPT AND ANSWER FORMATS

This appendix reproduces the text formats used in training and evaluation. Angle brackets mark fields that are filled in for each question, and long lines are wrapped to fit the page.

Chat Format

Training and serving use the same chat format, byte for byte:

```
<user>
question
</user>
<bot>
answer
</bot>
```

Answer Template

Every training answer follows this template:

```
According to <title or "CSULB Policy Statement NN-NN">,
section "...": <excerpt>
Source: <url>
```

Open-Book and Gold-Passage Prompt

The open-book training lane and the gold-passage evaluation condition place the source passage in front of the question under this header:

```
[Policy passage]
<title> (<policy number, if any>)
Section: <heading>
Date: <date>
Source: <url>
<passage, spaces collapsed, at most 1,500 characters>

[Question]
<question>
```

Reference Model Prompts

The reference model was claude-opus-5-5 at the low effort setting, with an output limit of 16,000 tokens and no temperature setting. The frozen closed-book prompt (version 1) was:

```
Answer this question from your own knowledge in 2 to 5 sentences,
then stop. Do not use a heading or a bullet list. Mention a policy
number or a source only if you know it.

Question: {question}
```

The amended closed-book prompt (version 2) adds one sentence at the start:

```
These questions are about policies of California State University,
Long Beach (CSULB). Answer this question from your own knowledge in
2 to 5 sentences, then stop. Do not use a heading or a bullet list.
Mention a policy number or a source only if you know it.

Question: {question}
```

The gold-passage prompt, which was not amended, was:

```
Answer this question only from the passage below, in 2 to 5
sentences, then stop. Do not use a heading or a bullet list. Mention
a policy number or a source only if the passage states it.

{gold_user}
```

In these prompts, {question} is the question text and {gold_user} is the same open-book prompt shown to the local model. In the actual prompts, each instruction is a single paragraph followed by a blank line and the question.

APPENDIX B

CHECKPOINT AND REPRODUCIBILITY RECORD

The code, data, logs, and evaluation files of this study are in the project repository (Quach, 2026), which builds on the original mini-AGI repository (Borsky, 2026). The weight folders are not in the repository because each folder contains a 436 MB optimizer file, GitHub rejects files larger than 100 MB, and the folders together occupy about 61 GB. They remain on the machine that trained them. Table 16 identifies each folder by its training step and by the SHA-256 hash of its core weight file; the full hashes are recorded in the repository's RESEARCH.md file.

TABLE 16. Checkpoint Folders and Core Weight Hashes

Checkpoint	Folder	Step	SHA-256 of core.npz (first 16 digits)
Base	weights/	914,508	913645b2b3942a04
65-minute	weights_csulb_65min	923,915	fb55119ba39ae2e1
Hour 2	weights_csulb_h2	950,140	688ce486fbe563ca
Hour 4	weights_csulb_h4	965,184	6cf3fa918f4f81a2
Hour 6	weights_csulb_h6	982,163	f6b9dd9d5801d71e
Hour 8	weights_csulb	995,953	ca52ded040b088fd

The evaluation can be traced through the following repository commits: the protocol and the 400-question sample were committed as 8085b11; the output and force options for the measurement tools as 2eea41f, which is also the code version used for every measurement; the frozen reference-model prompt as 57c3bc7; and the amendment that added the campus name as a4ddbb7. The main records are research/eval_protocol.md (the protocol and its amendment), research/eval/results.md (the result tables), research/end-to-end.md (the full account of the study), research/eval/questions_400.jsonl (the question sample), research/eval/local/answers.jsonl and research/eval/claude/ (all answers), research/eval/scores.json and research/eval/scores_v2.json (per-question scores), and research/eval/benchmarks/ (the frozen benchmark records).

A re-run of the training on the current version of the original code requires the equal lane schedule (the --lane-schedule equal option), because the developer's later default allocates reading time by expected improvement in held-out loss instead of giving every subject an equal share.

REFERENCES

REFERENCES

Allen-Zhu, Z., & Li, Y. (2023). *Physics of language models: Part 3.1, knowledge storage and extraction* (arXiv:2309.14316). arXiv. https://doi.org/10.48550/arXiv.2309.14316

Anthropic. (2026). *Claude Opus 5.5* [Large language model]. https://www.anthropic.com/claude

Aynetdinov, A., & Akbik, A. (2024). *SemScore: Automated evaluation of instruction-tuned LLMs based on semantic textual similarity* (arXiv:2401.17072). arXiv. https://doi.org/10.48550/arXiv.2401.17072

Banino, A., Balaguer, J., & Blundell, C. (2021). *PonderNet: Learning to ponder* (arXiv:2107.05407). arXiv. https://doi.org/10.48550/arXiv.2107.05407

Borsky, A. (2026). *mini-AGI* [Computer software]. GitHub. https://github.com/volotat/mini-AGI

Delétang, G., Ruoss, A., Duquenne, P.-A., Catt, E., Genewein, T., Mattern, C., Grau-Moya, J., Wenliang, L. K., Aitchison, M., Orseau, L., Hutter, M., & Veness, J. (2024). *Language modeling is compression* [Paper presentation]. Twelfth International Conference on Learning Representations, Vienna, Austria. https://doi.org/10.48550/arXiv.2309.10668

Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1383–1392). Association for Computational Linguistics. https://doi.org/10.18653/v1/P18-1128

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research, 23*(120), 1–39. https://jmlr.org/papers/v23/21-0998.html

French, R. M. (1999). Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences, 3*(4), 128–135. https://doi.org/10.1016/S1364-6613(99)01294-2

Graves, A. (2016). *Adaptive computation time for recurrent neural networks* (arXiv:1603.08983). arXiv. https://doi.org/10.48550/arXiv.1603.08983

Ibrahim, A., Thérien, B., Gupta, K., Richter, M. L., Anthony, Q., Lesort, T., Belilovsky, E., & Rish, I. (2024). Simple and scalable strategies to continually pre-train large language models. *Transactions on Machine Learning Research*. https://doi.org/10.48550/arXiv.2403.08763

Jang, J., Ye, S., Yang, S., Shin, J., Han, J., Kim, G., Choi, S. J., & Seo, M. (2022). *Towards continual knowledge learning of language models* [Paper presentation]. Tenth International Conference on Learning Representations, virtual event. https://doi.org/10.48550/arXiv.2110.03215

Kingma, D. P., & Ba, J. (2015). *Adam: A method for stochastic optimization* [Paper presentation]. Third International Conference on Learning Representations, San Diego, CA, United States. https://doi.org/10.48550/arXiv.1412.6980

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences, 114*(13), 3521–3526. https://doi.org/10.1073/pnas.1611835114

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems* (Vol. 33, pp. 9459–9474). Curran Associates.

Mahoney, M. (n.d.). *Large text compression benchmark*. Retrieved November 2, 2026, from https://mattmahoney.net/dc/text.html

McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. In G. H. Bower (Ed.), *Psychology of learning and motivation* (Vol. 24, pp. 109–165). Academic Press. https://doi.org/10.1016/S0079-7421(08)60536-8

Ovadia, O., Brief, M., Mishaeli, M., & Elisha, O. (2024). Fine-tuning or retrieval? Comparing knowledge injection in LLMs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (pp. 237–250). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-main.15

Petroni, F., Rocktäschel, T., Riedel, S., Lewis, P., Bakhtin, A., Wu, Y., & Miller, A. (2019). Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 2463–2473). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1250

Quach, P. T. (2026). *AGI-Research* [Computer software]. GitHub. https://github.com/quachphu/AGI-Research

Rae, J. W., Potapenko, A., Jayakumar, S. M., Hillier, C., & Lillicrap, T. P. (2020). *Compressive transformers for long-range sequence modelling* [Paper presentation]. Eighth International Conference on Learning Representations, virtual event. https://doi.org/10.48550/arXiv.1911.05507

Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing* (pp. 2383–2392). Association for Computational Linguistics. https://doi.org/10.18653/v1/D16-1264

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3982–3992). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410

Roberts, A., Raffel, C., & Shazeer, N. (2020). How much knowledge can you pack into the parameters of a language model? In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 5418–5426). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.437

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval, 3*(4), 333–389. https://doi.org/10.1561/1500000019

Shazeer, N. (2020). *GLU variants improve transformer* (arXiv:2002.05202). arXiv. https://doi.org/10.48550/arXiv.2002.05202

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). *Outrageously large neural networks: The sparsely-gated mixture-of-experts layer* [Paper presentation]. Fifth International Conference on Learning Representations, Toulon, France. https://doi.org/10.48550/arXiv.1701.06538

Song, K., Tan, X., Qin, T., Lu, J., & Liu, T.-Y. (2020). MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems* (Vol. 33). Curran Associates. https://proceedings.neurips.cc/paper/2020/hash/c3a690be93aa602ee2dc0ccab5b7b67e-Abstract.html

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing, 568,* Article 127063. https://doi.org/10.1016/j.neucom.2023.127063

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 5998–6008). Curran Associates.

Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., & Raffel, C. (2022). ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics, 10*, 291–306. https://doi.org/10.1162/tacl_a_00461

Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., Huang, F., & Zhou, J. (2025). *Qwen3 Embedding: Advancing text embedding and reranking through foundation models* (arXiv:2506.05176). arXiv. https://doi.org/10.48550/arXiv.2506.05176